

Form e controlli di input

I form sono il modo in cui una pagina raccoglie dati dall'utente: da una semplice ricerca a una registrazione completa.

■ Il tag `<form>`

Un form racchiude tutti i controlli di inserimento e definisce dove e come inviare i dati, tramite gli attributi `action` (l'indirizzo che riceve i dati) e `method` (di solito `GET` per richieste semplici o `POST` quando i dati sono più sensibili o numerosi).

```
<form action="/invia" method="POST">
  <!-- controlli di input qui dentro -->
</form>
```

■ I principali controlli di input

Il tag `<input>` cambia comportamento in base all'attributo `type`: testo semplice, email (con validazione automatica del formato), password (testo nascosto), numero, data, checkbox e radio button. Ogni controllo dovrebbe avere una `<label>` associata, per accessibilità e usabilità.

```
<label for="nome">Nome</label>
<input type="text" id="nome" name="nome" required />

<label for="email">Email</label>
<input type="email" id="email" name="email" required />

<p>Newsletter:</p>
<input type="checkbox" id="news" name="news" />
<label for="news">Iscrivimi alla newsletter</label>
```

L'attributo `for` della `<label>` deve corrispondere all'id del controllo: questo collegamento permette di cliccare sull'etichetta per attivare il campo, ed è essenziale per chi usa uno screen reader.

■ Select, textarea e invio

Per scegliere tra opzioni predefinite si usa `<select>` con dei `<option>`; per un testo lungo su più righe si usa `<textarea>`. Il form si invia con un `<button type="submit">` (o un `<input type="submit">`).

```
<label for="argomento">Argomento</label>
<select id="argomento" name="argomento">
  <option value="info">Informazioni</option>
  <option value="supporto">Supporto</option>
</select>

<label for="messaggio">Messaggio</label>
<textarea id="messaggio" name="messaggio" rows="4"></textarea>

<button type="submit">Invia</button>
```

■ Caso d'uso: un form di iscrizione con validazione nativa

HTML da solo, senza una riga di JavaScript, può già validare parte di un form: `required` impedisce l'invio se un campo obbligatorio è vuoto, `type="email"` verifica che il formato assomigli a un indirizzo email valido, e l'attributo `pattern` permette di richiedere un formato specifico tramite un'espressione regolare (ad esempio, un codice postale di 5 cifre).

```
<label for="cap">CAP</label>
<input
  type="text"
  id="cap"
  name="cap"
  pattern="[0-9]{5}"
  title="Inserisci un CAP di 5 cifre"
  required
/>
```

In un form di iscrizione a un corso, questa validazione nativa riduce già molti errori comuni (campo email senza @, CAP con lettere) prima ancora che i dati arrivino al server — che dovrà comunque validarli di nuovo, perché la validazione lato browser si può sempre aggirare, ma intanto offre un primo feedback immediato e gratuito all'utente.

ESERCIZI DI RIEPILOGO

1. Costruisci un form di contatto completo con campi Nome, Email, un menu a tendina "Argomento" con almeno 3 opzioni, un'area di testo per il messaggio e un pulsante di invio.
2. Aggiungi al form un gruppo di radio button per scegliere tra "Preferisco essere ricontattato per email" e "Preferisco essere ricontattato per telefono".
3. Spiega a parole la differenza pratica tra `method="GET"` e `method="POST"`, e quale useresti per un form di login. Motiva la risposta.